

기말 미니 과제 프로젝트

- 20240861 YC 한준탁

< 리눅스 gcc 컴파일 및 make 컴파일 >

```
linux@linux-VirtualBox:~/project03$ gcc -o project03 main.c monitor.c tui.c -lncurses
linux@linux-VirtualBox:~/project03$ ./project03
linux@linux-VirtualBox:~/project03$ gcc -o project03 main.c monitor.c tui.c -lncurses
linux@linux-VirtualBox:~/project03$ ./project03
linux@linux-VirtualBox:~/project03$ make
gcc -Wall -O2 -c main.c
gcc -Wall -O2 -c monitor.c
gcc -Wall -O2 -c tui.c
gcc -Wall -O2 -o mytop main.o monitor.o tui.o -lncurses
```

< monitor.h >

```
#ifndef MONITOR_H
#define MONITOR_H

/* 프로세스 정보 저장용 구조체 */
typedef struct {
    int pid;                // 프로세스 ID
    char name[256];         // 프로세스 이름
    char state;             // 실행 상태
    unsigned long utime;    // 사용자 모드 CPU 시간
    unsigned long stime;    // 커널 모드 CPU 시간
    long rss;              // 메모리 사용량
} ProcessInfo;

/* CPU 사용률 */
float get_cpu_usage();

/* 메모리 정보 */
void get_memory_info(long *total, long *used);

/* 프로세스 목록 */
int get_process_list(ProcessInfo *plist, int max_count);

#endif
```

< monitor.c >

```

#include "monitor.h"
#include <stdio.h> // fopen(), fscanf(), fclose()
#include <stdlib.h>
#include <string.h> // strcmp, strlen 문자열 처리 코드
#include <dirent.h> // opendir(), readdir()
#include <unistd.h> // 시스템 함수( ex : getpid() )

float get_cpu_usage() {
    FILE *fp;
    static unsigned long long prev_total = 0, prev_idle = 0;

    unsigned long long user,nice,system,idle,iowait,irq,softirq,steal;
    unsigned long long total;

    fp = fopen("/proc/stat", "r");
    if (!fp) return 0.0;

    int ret = fscanf(fp,
        "cpu %llu %llu %llu %llu %llu %llu %llu %llu",
        &user,&nice,&system,&idle,&iowait,&irq,&softirq,&steal
    );
    fclose(fp);

    if (ret != 8) return 0.0;

    total = user + nice + system + idle + iowait + irq + softirq + steal;

    if (prev_total == 0) {
        prev_total = total;
        prev_idle = idle;
        return 0.0;
    }

    float usage =
        (float)((total - prev_total) - (idle - prev_idle)) /
        (total - prev_total) * 100.0f;

    prev_total = total;
    prev_idle = idle;

    return usage;
}

```

```

/* 메모리 정보 */
void get_memory_info(long *total, long *used) {
    FILE *fp = fopen("/proc/meminfo", "r");
    if (!fp) return;

    char key[64];
    long mem_total = 0, mem_available = 0;

    while (fscanf(fp, "%63s", key) != EOF) {

        if (!strcmp(key, "MemTotal:")) {
            if (fscanf(fp, "%ld", &mem_total) != 1) break;
        }
        else if (!strcmp(key, "MemAvailable:")) {
            if (fscanf(fp, "%ld", &mem_available) != 1) break;
        }

        if (mem_total && mem_available) break;
    }

    fclose(fp);

    *total = mem_total / 1024;          // KB → MB

    *used = (mem_total - mem_available) / 1024;
}

/* 프로세스 목록 */
int get_process_list(ProcessInfo *plist, int max_count) {

    DIR *dp = opendir("/proc");
    if (!dp) return 0;

    struct dirent *entry;
    int count = 0;

    while ((entry = readdir(dp)) != NULL && count < max_count) {
        if (entry->d_name[0] < '0' || entry->d_name[0] > '9')
            continue;

        char stat_path[512], statm_path[512];

        snprintf(stat_path, sizeof(stat_path), "/proc/%s/stat", entry->d_name);
        snprintf(statm_path, sizeof(statm_path), "/proc/%s/statm", entry->d_name);

        FILE *fp = fopen(stat_path, "r");

```

```

        if (!fp) continue;

        ProcessInfo p;

        int ret = fscanf(fp,
            "%d (%255[^\n]) %c "
            "%*d %*d %*d %*d %*d %*d %*d %*d "
            "%lu %lu",
            &p.pid, p.name, &p.state,
            &p.utime, &p.stime
        );
        fclose(fp);

        if (ret != 5) continue;

        FILE *fm = fopen(statm_path, "r");
        if (fm) {
            long skip;
            if (fscanf(fm, "%ld %ld", &skip, &p.rss) != 2)
                p.rss = 0;
            fclose(fm);
        }

        plist[count++] = p;
    }

    closedir(dp);
    return count;
}

```

< tui.h >

```

#ifndef TUI_H
#define TUI_H

#include "monitor.h"

// TUI 초기화 및 종료
void init_tui();
void close_tui();

// 화면 출력
void draw_header(float cpu, long used, long total);
void draw_process_list(ProcessInfo *plist, int count, int start_index, int selected);

#endif

```

< tui.c >

```

#include "tui.h"
#include <ncurses.h>    // 화면 출력, 색상, 커서 제어

void init_tui() {
    initscr();          // ncurses 화면
    noecho();           // 입력한 글자 화면에 표시하지 않는 기능
    cbreak();           // 입력 즉시 처리
    curs_set(0);        // 커서 숨김
    keypad(stdscr, TRUE); // 방향키 사용
}

/* ncurses 종료 */
void close_tui() {
    endwin();
}

/* 화면 상단 CPU/메모리 표시 */
void draw_header(float cpu, long used, long total) {
    mvprintw(0, 0, "=====");
    mvprintw(1, 0, "                My-Top System Monitor                ");
    mvprintw(2, 0, "=====");
    mvprintw(3, 0, " CPU: %.2f %%    MEM: %ld MB / %ld MB (%.2f %%)",
        cpu, used, total, (used * 100.0) / total);

    mvprintw(4, 0, "-----");
}

/* 프로세스 목록 */
void draw_process_list(ProcessInfo *plist, int count, int start_index, int selected) {

    mvprintw(5, 0, " PID      CPU      MEM(KB)    STATE    NAME ");
    mvprintw(6, 0, "-----");

    int max_display = 15;

    for (int i = 0; i < max_display && (start_index + i) < count; i++) {
        int idx = start_index + i;

        if (i == selected) {
            attron(A_REVERSE); // 선택 강조
        }

        mvprintw(7 + i, 0,
            "%-8d %-8lu %-10ld %-8c %-20s",
            plist[idx].pid,
            plist[idx].utime + plist[idx].stime, // CPU 시간 합
            plist[idx].rss * 4,

            plist[idx].state,
            plist[idx].name);

        if (i == selected) {
            attroff(A_REVERSE);
        }
    }

    mvprintw(23, 0,
        " ↑/↓:스크롤  ←/→선택  F1/F2/F3:정렬  k:KILL s:STOP c:CONT q:종료 ");
}

```

< main.c >

```

#include "monitor.h"
#include "tui.h"
#include <unistd.h> // sleep()
#include <signal.h> // kill(), SIGSTOP, SIGKILL
#include <stdlib.h>
#include <ncurses.h> // 키 입력 처리

/* PID 정렬 */
int compare_pid(const void *a, const void *b) {
    return ((ProcessInfo*)a)->pid - ((ProcessInfo*)b)->pid;
}

/* CPU 정렬 */
int compare_cpu(const void *a, const void *b) {
    long aa = ((ProcessInfo*)a)->utime + ((ProcessInfo*)a)->stime;
    long bb = ((ProcessInfo*)b)->utime + ((ProcessInfo*)b)->stime;
    return bb - aa;
}

/* 메모리 정렬 */
int compare_mem(const void *a, const void *b) {
    return ((ProcessInfo*)b)->rss - ((ProcessInfo*)a)->rss;
}

```

```

int main() {
    init_tui();

    int start = 0;
    int selected = 0;
    int sort_mode = 0;

    while (1) {

        float cpu = get_cpu_usage();
        long total, used;
        get_memory_info(&total, &used);

        ProcessInfo plist[300];
        int count = get_process_list(plist, 300);

        if (sort_mode == 0) qsort(plist, count, sizeof(ProcessInfo), compare_pid);
        if (sort_mode == 1) qsort(plist, count, sizeof(ProcessInfo), compare_cpu);
        if (sort_mode == 2) qsort(plist, count, sizeof(ProcessInfo), compare_mem);

        clear();
        draw_header(cpu, used, total);
        draw_process_list(plist, count, start, selected);
    }
}

```

```

refresh();

timeout(0);
int ch = getch();

if (ch == 'q') break;

/* 스크롤 */
if (ch == KEY_DOWN && start < count - 1) start++;
if (ch == KEY_UP && start > 0) start--;

/* 선택 이동 */
if (ch == KEY_RIGHT && selected < 14) selected++;
if (ch == KEY_LEFT && selected > 0) selected--;

/* 정렬 */
if (ch == KEY_F(1)) sort_mode = 0;
if (ch == KEY_F(2)) sort_mode = 1;
if (ch == KEY_F(3)) sort_mode = 2;

/* 프로세스 제어 */
int idx = start + selected;
if (idx < count) {
    int pid = plist[idx].pid;

    if (ch == 'k') kill(pid, SIGKILL);
    if (ch == 's') kill(pid, SIGSTOP);
    if (ch == 'c') kill(pid, SIGCONT);
}

sleep(1);
}

close_tui();
return 0;
}

```

< Makefile >

```

CC = gcc
CFLAGS = -Wall -O2
LIBS = -lnurses

all: mytop

mytop: main.o monitor.o tui.o
    $(CC) $(CFLAGS) -o mytop main.o monitor.o tui.o $(LIBS)

main.o: main.c monitor.h tui.h
    $(CC) $(CFLAGS) -c main.c

monitor.o: monitor.c monitor.h
    $(CC) $(CFLAGS) -c monitor.c

tui.o: tui.c tui.h monitor.h
    $(CC) $(CFLAGS) -c tui.c

clean:
    rm -f *.o mytop

```

< 실행결과 >

```
=====
My-Top System Monitor
=====
CPU: 6.30 %    MEM: 2470 MB / 3915 MB (63.09 %)
-----
PID      CPU      MEM(KB)  STATE   NAME
-----
1        3667    14436    S       systemd
2         0         0        S       kthreadd
3         0         0        S       pool_workqueue_release
4         0         0        I       kworker/R-rcu_gp
5         0         0        I       kworker/R-sync_wq
6         0         0        I       kworker/R-kvfree_rcu_reclaim
7         0         0        I       kworker/R-slub_flushwq
8         0         0        I       kworker/R-netns
11        0         0        I       kworker/0:0H-events_highpri
13        0         0        I       kworker/R-mm_percpu_wq
14        0         0        I       rcu_tasks_kthread
15        0         0        I       rcu_tasks_rude_kthread
16        0         0        I       rcu_tasks_trace_kthread
17        0         0        S       ksoftirqd/0
18        0         0        I       rcu_preempt

~F~Q/~F~S:~J~A를 ~F~P/~F~R:~D~C~] F1/F2/F3:~U렬 k:KILL s:STOP c:CONT q::~E~L
```

```
=====
My-Top System Monitor
=====
CPU: 5.30 %    MEM: 2411 MB / 3915 MB (61.58 %)
-----
PID      CPU      MEM(KB)  STATE   NAME
-----
4         0         0        I       kworker/R-rcu_gp
5         0         0        I       kworker/R-sync_wq
6         0         0        I       kworker/R-kvfree_rcu_reclaim
7         0         0        I       kworker/R-slub_flushwq
8         0         0        I       kworker/R-netns
11        0         0        I       kworker/0:0H-events_highpri
13        0         0        I       kworker/R-mm_percpu_wq
14        0         0        I       rcu_tasks_kthread
15        0         0        I       rcu_tasks_rude_kthread
16        0         0        I       rcu_tasks_trace_kthread
17        0         0        S       ksoftirqd/0
18        0         0        I       rcu_preempt
19        0         0        S       rcu_exp_par_gp_kthread_worker/0
20        0         0        S       rcu_exp_gp_kthread_worker
21        0         0        S       migration/0

~F~Q/~F~S:~J~A를 ~F~P/~F~R:~D~C~] F1/F2/F3:~U렬 k:KILL s:STOP c:CONT q::~E~L
```



```
=====
My-Top System Monitor
=====
CPU: 6.08 %    MEM: 2422 MB / 3915 MB (61.86 %)
-----
PID      CPU      MEM(KB)  STATE   NAME
-----
2108     457007   275164   S       gnome-shell
1        3749     14440    S       systemd
3036     1842     688404   S       firefox
1868     1509     12436    S       systemd
6553     531      5264     S       bash
2759     332      22280    S       tracker-miner-f
918      315      26108    S       snapd
1464     140      14508    S       colord
2447     125      29084    S       ibus-extension-
2208     121      14652    S       gnome-shell-cal
1001     109      17580    S       NetworkManager
3231     97       28244    S       forkserver
2441     96       10416    S       gvfs-udisks2-vo
2263     94       19804    S       gsd-keyboard
2378     88       20080    S       gsd-wacom

~F~/~F~S:~J~A를 ~F~P/~F~R:~D~C~] F1/F2/F3:~U열 k:KILL s:STOP c:CONT q:~E~L
```

```
=====
My-Top System Monitor
=====
CPU: 11.86 %    MEM: 2437 MB / 3915 MB (62.25 %)
-----
PID      CPU      MEM(KB)  STATE   NAME
-----
5568     37       803276   S       Isolated Web Co
3036     1842     688448   S       firefox
3829     76       665456   S       Isolated Web Co
2108     459754   275168   S       gnome-shell
3306     14       131476   S       Privileged Cont
3262     9        89380    S       WebExtensions
8690     3        84024    S       Isolated Web Co
14081    0        78716    S       Isolated Servic
4057     0        76888    S       Isolated Servic
6545     10       73116    S       gnome-terminal-
14199    0        67316    S       Web Content
14113    0        67312    S       Web Content
14147    0        67260    S       Web Content
13542    0        65772    S       gjs
2326     51       63224    S       evolution-alarm

~F~/~F~S:~J~A를 ~F~P/~F~R:~D~C~] F1/F2/F3:~U열 k:KILL s:STOP c:CONT q:~E~L
```

< 미니 프로젝트 구조 >

< main.c >

1. CPU/메모리/프로세스 데이터 요청 (monitor.c)
2. 화면 출력 (tui.c)
3. 키보드 입력(정렬, 스크롤 등)

4. 1초마다 반복

< monitor.c (데이터 수집) >

1. /proc 파일 시스템을 읽어옴
2. CPU 사용률 -> get_cpu_usage()
3. 메모리 사용량 -> get_memory_info()
4. 프로세스 목록 -> get_process_list()

< tui.c (UI 출력) >

1. ncurses 라이브러리를 사용해서 UI 화면 출력

< 파일 시스템 데이터 참조 >

1. /proc/stat (CPU 정보)
 - user, nice, system, idle, iowait, irq, softirq, steal 값을 불러와 CPU 누적값 계산
 - 두 시점의 값을 비교해서 CPU 사용률 계산
2. /proc/meminfo (메모리 정보)
 - Used = MemTotal - MemAvailable -> 메모리 사용량 계산
3. /proc/[pid]/stat (프로세스 정보)
 - pid, comm, state, utime, stime을 파싱해서 프로세스 이름, 상태, CPU 사용량 출력
4. /proc/[pid]/statm (프로세스 메모리 정보)
 - RSS를 읽어 메모리 점유량 계산
5. /proc (프로세스)
 - 숫자로 된 디렉토리를 스캔해서 PID 목록 불러옴

< 파일별로 중요한 코드 설명 >

< monitor.c >

< CPU 사용률 계산 부분 >

```
fp = fopen("/proc/stat", "r");

fscanf(fp,

"cpu %llu %llu %llu %llu %llu %llu %llu %llu",

&user, &nice, &system, &idle, &iowait, &irq, &softirq, &steal);

float usage = (float)((total - prev_total) - (idle - prev_idle)) / (total - prev_total) *
100.0f;
```

- /proc/stat 내용을 읽어 CPU 사용률 계산 (시스템 전체 CPU 시간 누적값)

< 메모리 정보 함수 >

```
FILE *fp = fopen("/proc/meminfo", "r");

if (!strcmp(key, "MemTotal:"))

    fscanf(fp, "%ld", &mem_total);

*total = mem_total / 1024;

*used = (mem_total - mem_available) / 1024;
```

- /proc/meminfo 메모리 정보 제공
- 문자열 비교해서 MemTotal 찾기
- KB 단위를 MB로 변경

< 프로세스 목록 수집 함수 >

```
DIR *dp = opendir("/proc");

while ((entry = readdir(dp)) != NULL) {

    if (entry->d_name[0] < '0' || entry->d_name[0] > '9')

        continue;

    fscanf(fp,

"%d (%255[^\n]) %c "

"%*d %*d %*d %*d %*d %*d %*d %*d "

"%lu %lu",

&p.pid, p.name, &p.state, &p.utime, &p.stime );
```

- proc 디렉터리 읽기
- proc 안에 숫자로 시작하는 디렉토리는 프로세스이어서 필터링
- (%255[^\n]) 괄호 안의 프로그램 이름을 끝까지 읽는 식

< tui.h >

- ncurses UI 함수

< tui.c >

initscr(); -> ncurses 모드

noecho(); -> 키 입력을 화면에 출력 X

cbreak(); -> 버퍼링 없이 입력 처리

`curs_set(0);` -> 커서 숨김

`keypad(stdscr, TRUE);` -> 방향키 입력

`mvprintw(1, 0, " CPU: %.2f%% MEM: %ldMB / %ldMB (%.2f%%)");`

- CPU/메모리 출력

`if (i == selected)`

`attron(A_REVERSE);`

- 프로세스 목록 출력

< main.c (메인 함수) >

`while (1) {`

`float cpu = get_cpu_usage();`

`long total, used;`

`get_memory_info(&total, &used);`

`ProcessInfo plist[300];`

`int count = get_process_list(plist, 300);`

`if (sort_mode == 0) qsort(plist, count, sizeof(ProcessInfo), compare_pid);`

`if (sort_mode == 1) qsort(plist, count, sizeof(ProcessInfo), compare_cpu);`

`if (sort_mode == 2) qsort(plist, count, sizeof(ProcessInfo), compare_mem);`

`clear();`

`draw_header(cpu, used, total);`

```

draw_process_list(plist, count, start, selected); refresh();

timeout(0);

int ch = getch();

if (ch == 'q') break;

/* 스크롤 */

if (ch == KEY_DOWN && start < count - 1) start++;

if (ch == KEY_UP && start > 0) start--;

/* 선택 이동 */

if (ch == KEY_RIGHT && selected < 14) selected++;

if (ch == KEY_LEFT && selected > 0) selected--;

/* 정렬 */

if (ch == KEY_F(1)) sort_mode = 0;

if (ch == KEY_F(2)) sort_mode = 1;

if (ch == KEY_F(3)) sort_mode = 2;

/* 프로세스 제어 */

int idx = start + selected;

if (idx < count) {

    int pid = plist[idx].pid;

    if (ch == 'k') kill(pid, SIGKILL);

    if (ch == 's') kill(pid, SIGSTOP);

    if (ch == 'c') kill(pid, SIGCONT);

```

```
}
```

```
sleep(1);
```

```
}
```

- 1초마다 새로운 값을 받아 화면 갱신
- F1, F2, F3 입력으로 정렬 기준 변경
- 방향키 입력/화면 스크롤
- 프로세스 제어 (로직은 잘 구현했으나 커널 스레드여서 리눅스 우분투 환경에서 동작 X)

↑ ↓ : 스크롤

← → : 선택 커서 이동

F1 : PID 정렬

F2 : CPU 정렬

F3 : MEM 정렬

k : kill

s : stop

c : continue